HealthPal 技术框架文档

整理时间： 2026-03-10
项目启动： 2026-03-08
当前阶段： Phase 1 - MVP 开发（第 1-3 月）

一、项目概述

HealthPal 是一款基于 AI 技术的个人健康管理工具，帮助用户整合、分析和管理个人及家庭健康数据。

核心功能： - 健康文档上传 + OCR 识别 - 健康指标趋势分析 - 用药提醒系统 - AI 健康分析报告 - 家庭多成员档案管理

二、整体技术架构

```
┌──────────────
```

```
                        │
┌──────────────────────────────────────────▼──────────────────────────────────────────
│              应用服务层 (Application Layer)            │
│              Python FastAPI + Uvicorn                    │
│   ┌──────────────────────┐  ┌──────────────────────┐  ┌──────────────────────────
│   │  用户服务   │ │  档案服务   │ │   AI 分析服务    │    │
│  │  Auth       │ │  Records    │ │   Analysis      │    │
│   └──────────────────────┘  └──────────────────────┘  └──────────────────────────
│   ┌──────────────────────┐  ┌──────────────────────┐  ┌──────────────────────────
│   │  指标服务   │ │  提醒服务   │ │   OCR 服务       │    │
│  │  Indicators │ │  Medications│ │   OCR Engine    │    │
│   └──────────────────────┘  └──────────────────────┘  └──────────────────────────
└──────────────────────────────────────────┴──────────────────────────────────────────
                        │
┌──────────────────────────────────────────▼──────────────────────────────────────────
│                 数据持久层 (Data Layer)                │
│   ┌──────────────────────┐  ┌──────────────────────┐  ┌──────────────────────────
│  │   MySQL     │ │  MongoDB    │ │      Redis       │    │
│   │  关系型数据  │ │  文档型数据  │ │   缓存/会话      │    │
│   │  8.0        │ │  6.0        │ │   7.0           │    │
│   └──────────────────────┘  └──────────────────────┘  └──────────────────────────
└──────────────────────────────────────────────────────────────────────────────────────
```

三、技术栈详情

3.1 后端技术栈

模块	技术选型	版本	说明
开发语言	Python	3.11+	主开发语言

模块	技术选型	版本	说明
Web 框架	FastAPI	0.109+	高性能异步 API
ASGI 服务器	Uvicorn	0.27+	ASGI 服务器
ORM	SQLAlchemy	2.0+	数据库 ORM
数据验证	Pydantic	2.0+	数据验证和序列化
认证	PyJWT	2.8+	JWT Token 管理
密码加密	Passlib	1.7+	密码哈希 (bcrypt)
数据库迁移	Alembic	1.13+	数据库版本管理
任务队列	Celery	5.3+	异步任务 (OCR/AI)
消息代理	Redis	7.0	Celery Broker

3.2 移动端技术栈

模块	技术选型	版本	说明
框架	Flutter	3.x	跨平台 UI 框架
语言	Dart	3.x	移动端开发语言
状态管理	Provider/Riverpod	6.0+/2.4+	状态管理方案
网络请求	Dio + Retrofit	5.3+/4.0+	HTTP 客户端
本地存储	Hive	2.2+	轻量级 NoSQL 存储
图片处理	Image Picker	1.0+	拍照/相册选择
图表	fl_chart	0.64+	折线图/柱状图

模块	技术选型	版本	说明
推送通知	Flutter Local Notifications	16.1+	本地推送
权限管理	Permission Handler	11.0+	运行时权限

3.3 数据存储技术栈

数据库	版本	用途	部署方式
MySQL	8.0	用户/档案/指标/提醒等关系型数据	Docker
MongoDB	6.0	用户行为日志/AI 分析结果/设备缓存	Docker
Redis	7.0	Session/Token 黑名单/验证码/热点缓存/限流	Docker

3.4 第三方服务

服务	提供商	用途
OCR 识别	百度 OCR / 腾讯 OCR	体检报告/病历文字识别
AI 分析	(待定)	健康风险评估/指标解读
短信服务	(待定)	验证码/提醒通知
对象存储	(待定)	健康文档图片存储

3.5 运维与部署

工具	用途
Docker	容器化部署
Docker Compose	本地开发环境编排

工具	用途
Kubernetes	生产环境容器编排 (Phase 2)
GitHub Actions	CI/CD 流水线
Nginx	反向代理/负载均衡
Prometheus + Grafana	监控告警 (Phase 2)

四、项目目录结构

```
HealthPal/
├── config/                      # 配置文件
│   ├── docker-compose.yml       # MySQL+MongoDB+Redis
│   ├── nginx.conf               # Nginx 配置
│   └── k8s/                     # Kubernetes 配置 (Phase 2)
│
├── docs/                        # 项目文档
│   ├── README.md                # 项目说明
│   ├── API.md                   # API 接口文档
│   ├── DB_DESIGN.md             # 数据库设计
│   ├── TASK_BREAKDOWN.md        # 任务分解
│   └── MOBILE_DEV_PLAN.md       # 移动端开发计划
│
├── src/
│   ├── backend/                 # Python 后端
│   │   ├── app/
│   │   │   ├── main.py          # 应用入口
│   │   │   ├── config.py        # 配置管理
│   │   │   ├── models/          # 数据模型 (SQLAlchemy)
│   │   │   ├── schemas/         # Pydantic Schema
```

```
│   │   │   ├── api/             # API 路由
│   │   │   │   ├── auth.py      # 认证接口
│   │   │   │   ├── users.py     # 用户接口
│   │   │   │   ├── records.py   # 档案接口
│   │   │   │   ├── indicators.py# 指标接口
│   │   │   │   └── medications.py# 用药接口
│   │   │   ├── services/        # 业务逻辑层
│   │   │   │   ├── auth_service.py
│   │   │   │   ├── ocr_service.py
│   │   │   │   └── ai_service.py
│   │   │   ├── db/              # 数据库连接
│   │   │   └── utils/           # 工具函数
│   │   ├── tests/               # 单元测试
│   │   ├── scripts/             # 脚本工具
│   │   ├── requirements.txt     # Python 依赖
│   │   └── setup.sh             # 环境搭建脚本
│   │
│   └── mobile/                  # Flutter 移动端
│       ├── lib/
│       │   ├── main.dart        # 应用入口
│       │   ├── app.dart         # App 配置
│       │   ├── config/          # 配置
│       │   │   ├── routes.dart  # 路由
│       │   │   └── theme.dart   # 主题
│       │   ├── models/          # 数据模型
│       │   ├── services/        # 服务层
│       │   ├── providers/       # 状态管理
│       │   ├── screens/         # 页面
│       │   │   ├── home/        # 首页
│       │   │   ├── records/     # 档案管理
│       │   │   ├── analysis/    # 分析报告
│       │   │   └── profile/     # 个人中心
```

```
│       │   ├── widgets/         # 通用组件
│       │   └── utils/           # 工具类
│       ├── assets/              # 资源文件
│       ├── test/                # 测试
│       └── pubspec.yaml         # Flutter 依赖
│
└── tests/                       # 集成测试
```

五、数据库设计

5.1 MySQL 表结构

表名	说明	关键字段
users	用户表	uuid, phone, password_hash, nickname, avatar
family_members	家庭成员表	user_id, name, relation, gender, birthday
health_records	健康档案表	user_id, member_id, record_type, ocr_status, structured_data(JSON)
health_indicators	健康指标表	record_id, indicator_code, value, unit, status
medication_reminders	用药提醒表	user_id, medicine_name, dosage, reminder_times(JSON)

表名	说明	关键字段
reminder_logs	提醒记录表	reminder_id, scheduled_time, status

5.2 MongoDB 集合

集合名	说明
user_behaviors	用户行为日志 (view/upload/search)
ai_analyses	AI 分析结果缓存
device_data_cache	设备数据缓存 (血压计/血糖仪)

5.3 Redis Key 设计

```
# Session 和认证
session:{uuid}                  # 用户会话
token:blacklist:{jti}            # Token 黑名单

# 验证码
sms:code:{phone}                # 短信验证码
email:code:{email}               # 邮箱验证码

# 热点数据缓存
user:profile:{userId}            # 用户资料
record:detail:{recordId}         # 档案详情

# 分布式锁
lock:upload:{userId}             # 上传锁
lock:ocr:{recordId}              # OCR 处理锁
```

```
# 限流计数
ratelimit:api:{userId}:{endpoint} # API 限流
```

六、API 设计

6.1 接口规范

- **基础 URL:** `https://api.healthpal.com/v1`
- **认证方式:** JWT Bearer Token
- **响应格式:** JSON
- **字符编码:** UTF-8

6.2 核心接口分类

分类	接口路径	说明
认证	POST /auth/login, /auth/register	登录/注册
用户	GET/PUT /user/profile	用户资料管理
家庭成员	GET/POST /family/members	家庭成员管理
健康档案	GET/POST/DELETE /records	档案上传/查询/删除
健康指标	GET /indicators/trend, POST /indicators	指标趋势/手动录入
用药提醒	GET/POST /medications/reminders	提醒管理
AI 分析	GET /ai/analysis/report	健康分析报告

6.3 限流策略

接口类型	限流	说明
认证接口	5 次/分钟	防止暴力破解
上传接口	10 次/分钟	防止滥用
普通查询	100 次/分钟	正常查询
AI 分析	20 次/小时	计算密集型

七、安全设计

7.1 认证安全

- **密码存储:** bcrypt 加密哈希
- **Token 机制:** JWT + Refresh Token
- **Token 有效期:** 7 天
- **Token 黑名单:** Redis 存储已注销 Token

7.2 数据安全

- **传输加密:** HTTPS (TLS 1.3)
- **敏感数据:** 手机号脱敏展示
- **文件存储:** 私有 Bucket + 签名 URL

7.3 接口安全

- **限流保护:** Redis 计数器限流
- **SQL 注入防护:** SQLAlchemy ORM 参数化
- **XSS 防护:** 输入验证 + 输出转义

八、开发里程碑

Phase 1: MVP 开发（第 1-3 月）

阶段	时间	主要任务	完成度
技术架构搭建	第 1 月	后端框架、数据库设计、认证系统	80%
移动端基础	第 1-2 月	Flutter 环境、UI 组件、网络封装	0%
核心功能	第 2-3 月	文档上传、OCR、档案展示	0%
测试部署	第 3 月	单元测试、集成测试、云服务器部署	0%

Phase 2: 功能完善（第 4-6 月）

- AI 分析功能（指标解读 + 风险评估）
- 智能提醒系统（用药/复查/体检）
- 趋势分析图表
- Beta 测试（100 名种子用户）

Phase 3: 高级功能（第 7-9 月）

- 家庭健康管理（多成员档案）
- 手动录入健康数据（血压/血糖/体重）
- AI 分析 2.0（风险预测模型）

Phase 4: 商业化（第 10-12 月）

- 付费功能上线（订阅系统）
- B 端 API 开放（体检机构对接）
- 运营数据看板

九、当前进度

已完成

- [x] 项目立项和 PRD 确认
- [x] 技术架构设计
- [x] 数据库设计
- [x] API 接口设计
- [x] 后端框架搭建 (FastAPI)
- [x] 用户认证系统 (JWT)

待开发

- [ ] 数据库迁移脚本 (Alembic)
- [ ] OCR 服务集成
- [ ] Flutter 移动端开发
- [ ] 健康档案上传功能
- [ ] AI 分析服务

十、风险与应对

风险	影响	应对措施
OCR 识别准确率	高	选择成熟服务商 (百度/腾讯)，支持人工校正
医疗数据合规	高	咨询法律顾问，符合《个人信息保护法》要求
AI 分析准确性	中	明确免责声明，仅提供参考建议

风险	影响	应对措施
跨平台兼容性问题	中	充分测试主流机型，建立兼容性矩阵

文档维护： 每周五同步更新

负责人： 嵌入式工程师 Agent